

DAY 2



UNDERSTANDING EMBEDDINGS

Types of Embeddings



There are many types of embeddings, like word embeddings and sentence embeddings that are used to indicate different levels of abstractions (word versus sentence), as illustrated in Figure 1-10.

Bag-of-words, for instance, creates embeddings at a document level since it represents the entire document. In contrast, word2vec generates embeddings for words only. Throughout the course, embeddings will take on a central role as they are utilized in many use cases, such as classification, clustering, and semantic search and retrieval-augmented generation, we will take our first deep dive into token embeddings.

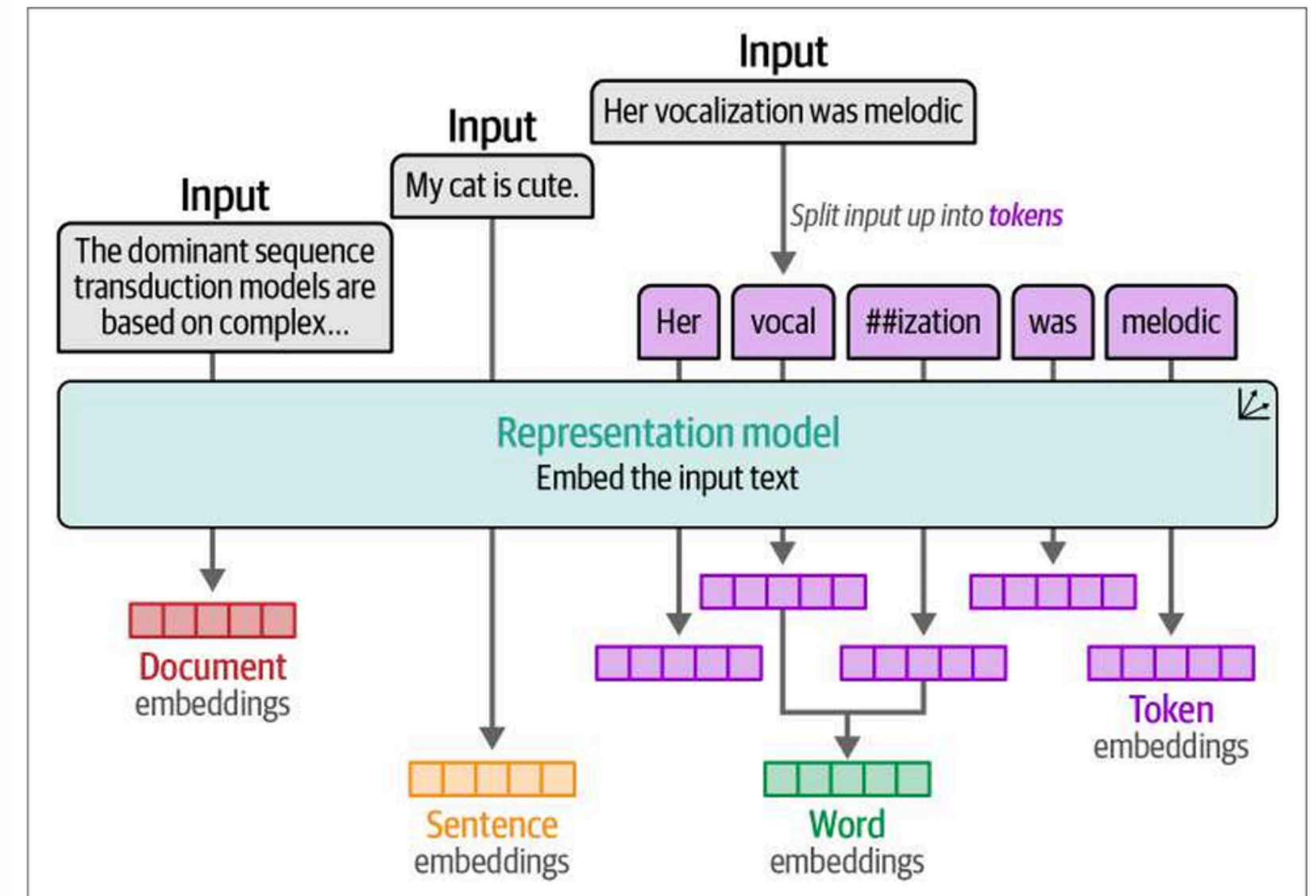


Figure 1-10. Embeddings can be created for different types of input.

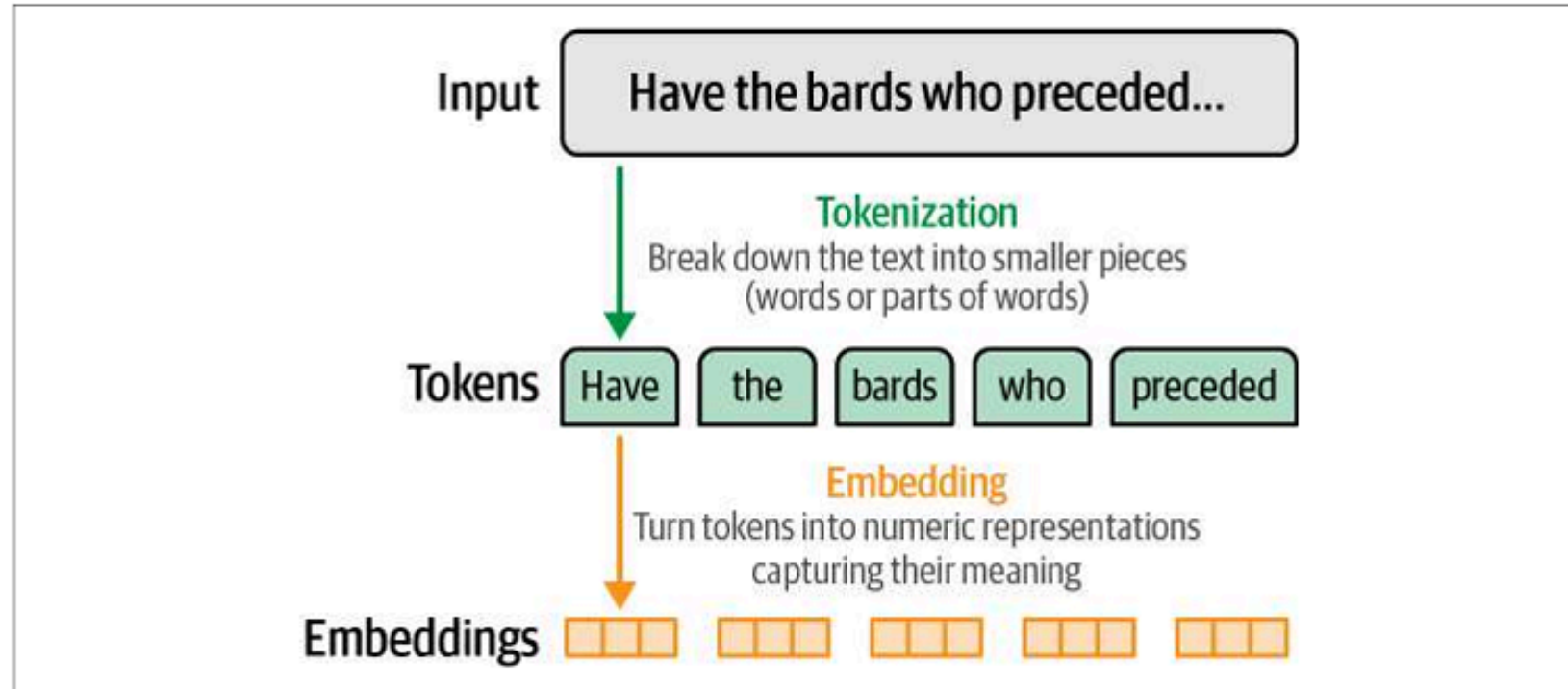


Figure 2-1. Language models deal with text in small chunks called tokens. For the language model to compute language, it needs to turn tokens into numeric representations called embeddings.

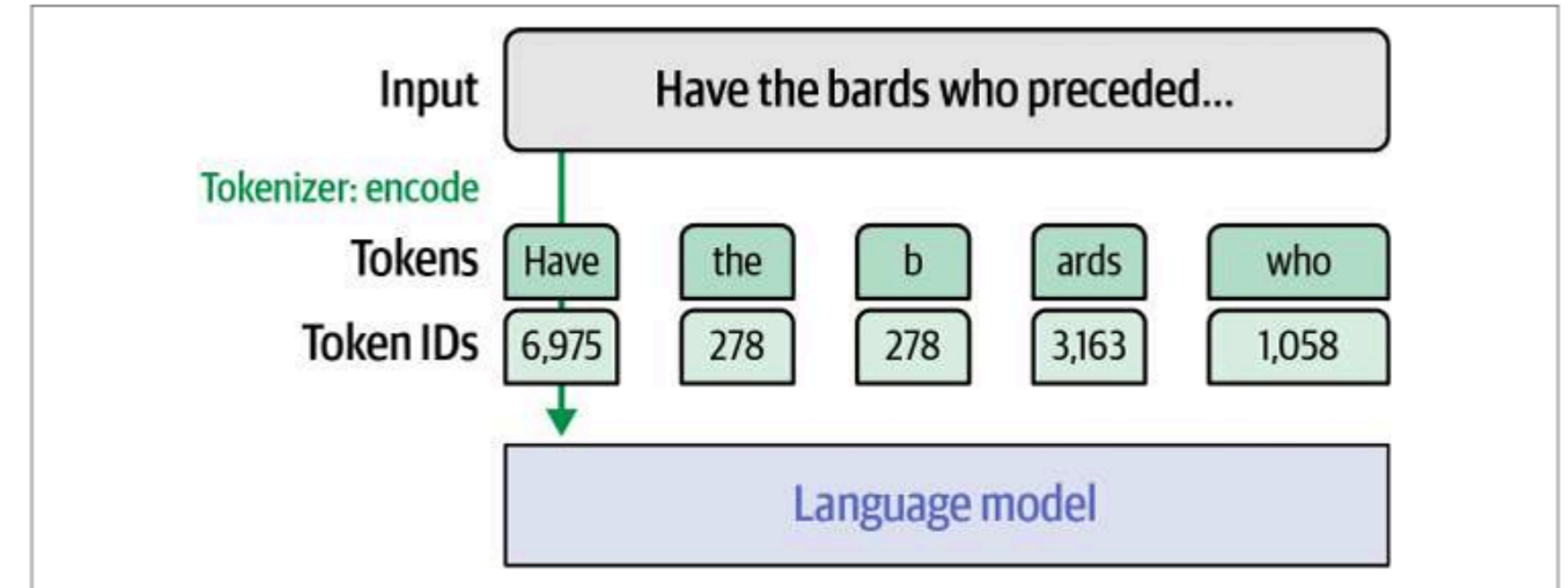


Figure 2-4. A tokenizer processes the input prompt and prepares the actual input into the language model: a list of token IDs. The specific token IDs in the figure are just demonstrative.

Encoding and Decoding Context with Attention



- The training process of word2vec creates static, downloadable representations of words. For instance, the word “bank” will always have the same embedding regardless of the context in which it is used. However, “bank” can refer to both a financial bank as well as the bank of a river. Its meaning, and therefore its embeddings, should change depending on the context.
- A step in encoding this text was achieved through **recurrent neural networks (RNNs)**. These are variants of neural networks that can model sequences as an additional input.
- To do so, these RNNs are used for two tasks, encoding or representing an input sentence and decoding or generating an output sentence.

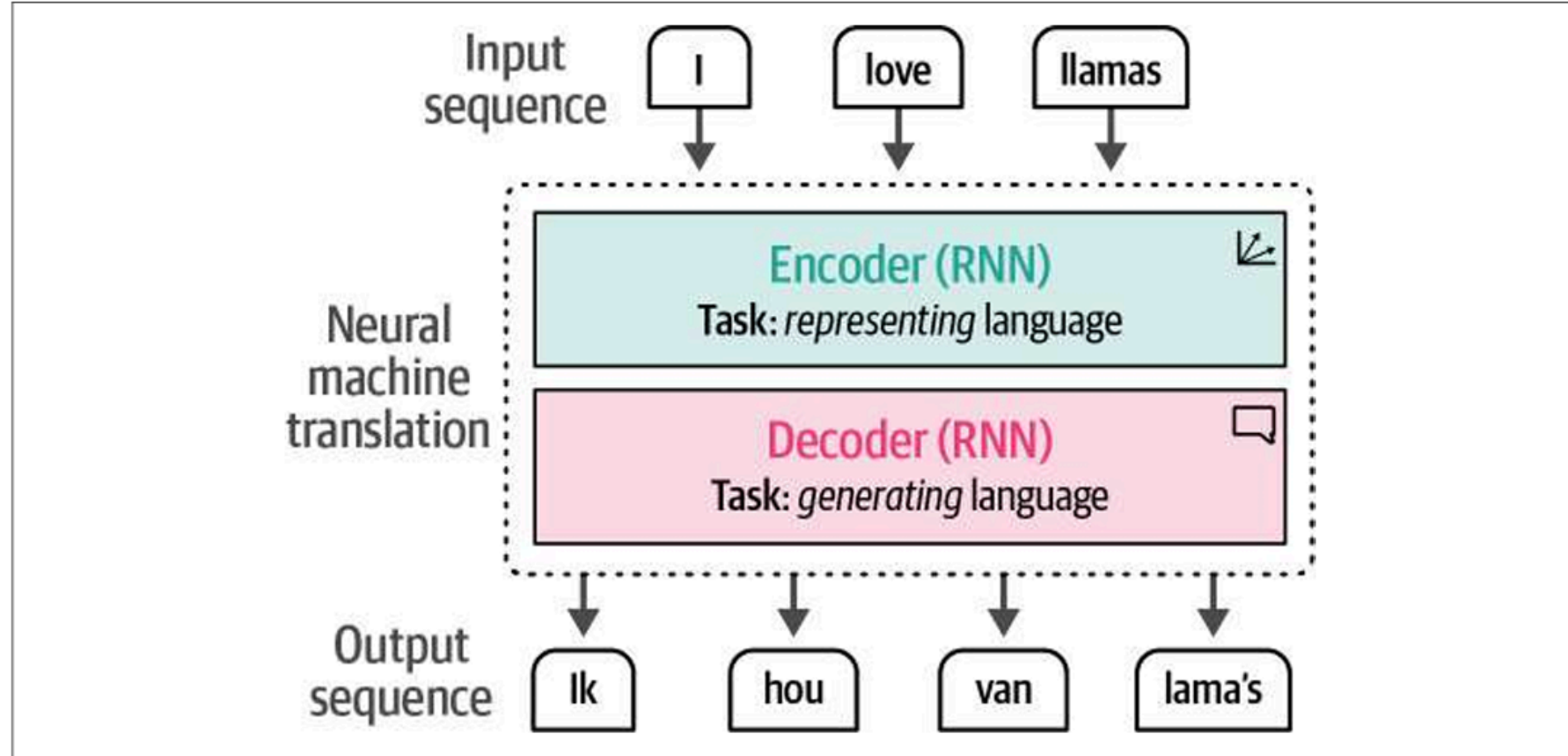


Figure 1-11. Two recurrent neural networks (decoder and encoder) translating an input sequence from English to Dutch.

Figure 1-11 illustrates this concept by showing how a sentence like “I love llamas” gets translated to the Dutch “Ik hou van lama’s.”

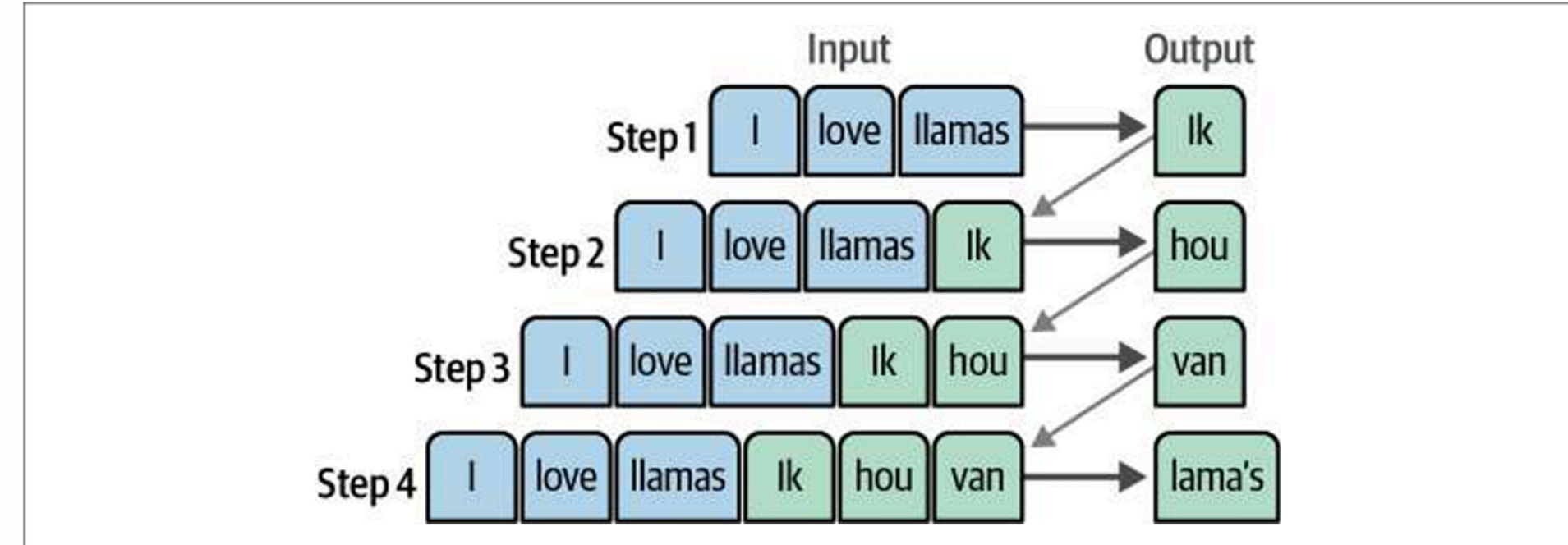


Figure 1-12. Each previous output token is used as input to generate the next token.

Each step in this **architecture is autoregressive**. When generating the next word, this architecture needs to consume all previously generated words, as shown in Figure 1-12.

The encoding step aims to represent the input as well as possible, generating the context in the form of an embedding, which serves as the input for the decoder. To generate this representation, it takes embeddings as its inputs for words, which means we can use word2vec for the initial representations. In Figure 1-13, we can observe this process. Note how the inputs are processed sequentially, one at a time, as well as the output.

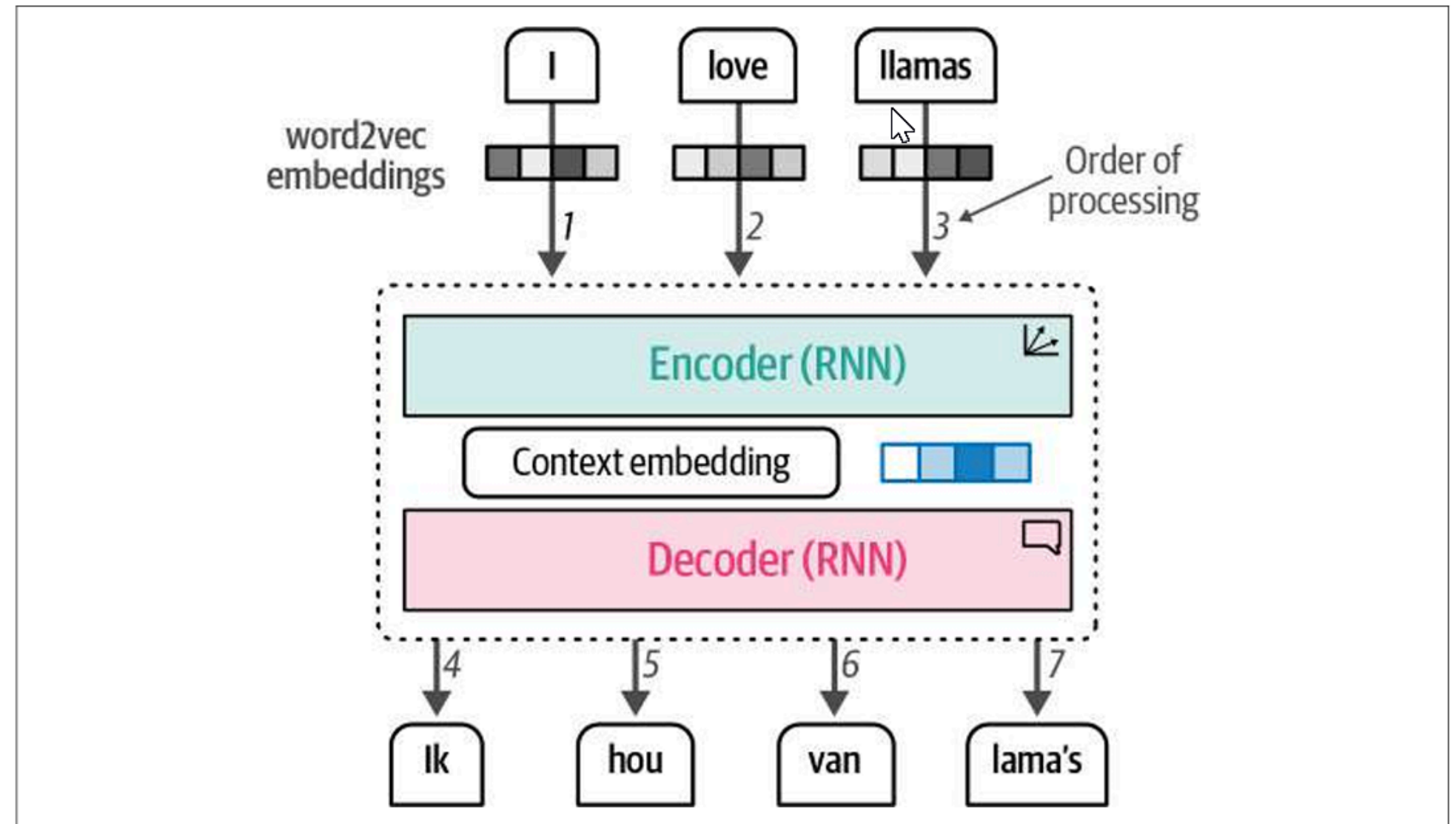


Figure 1-13. Using word2vec embeddings, a context embedding is generated that represents the entire sequence.

Context Embedding = A summary of the whole sentence.

- This context embedding, however, makes it difficult to deal with longer sentences since it is merely a single embedding representing the entire input. In 2014, a solution called attention was introduced that highly improved upon the original architecture.
- Attention allows a model to focus on parts of the input sequence that are relevant to one another (“attend” to each other) and amplify their signal, as shown in Figure 1-14. Attention selectively determines which words are most important in a given sentence. For instance, the output word “lama’s” is Dutch for “llamas,” which is why the attention between both is high. Similarly, the words “lama’s” and “I” have lower attention since they aren’t as related.

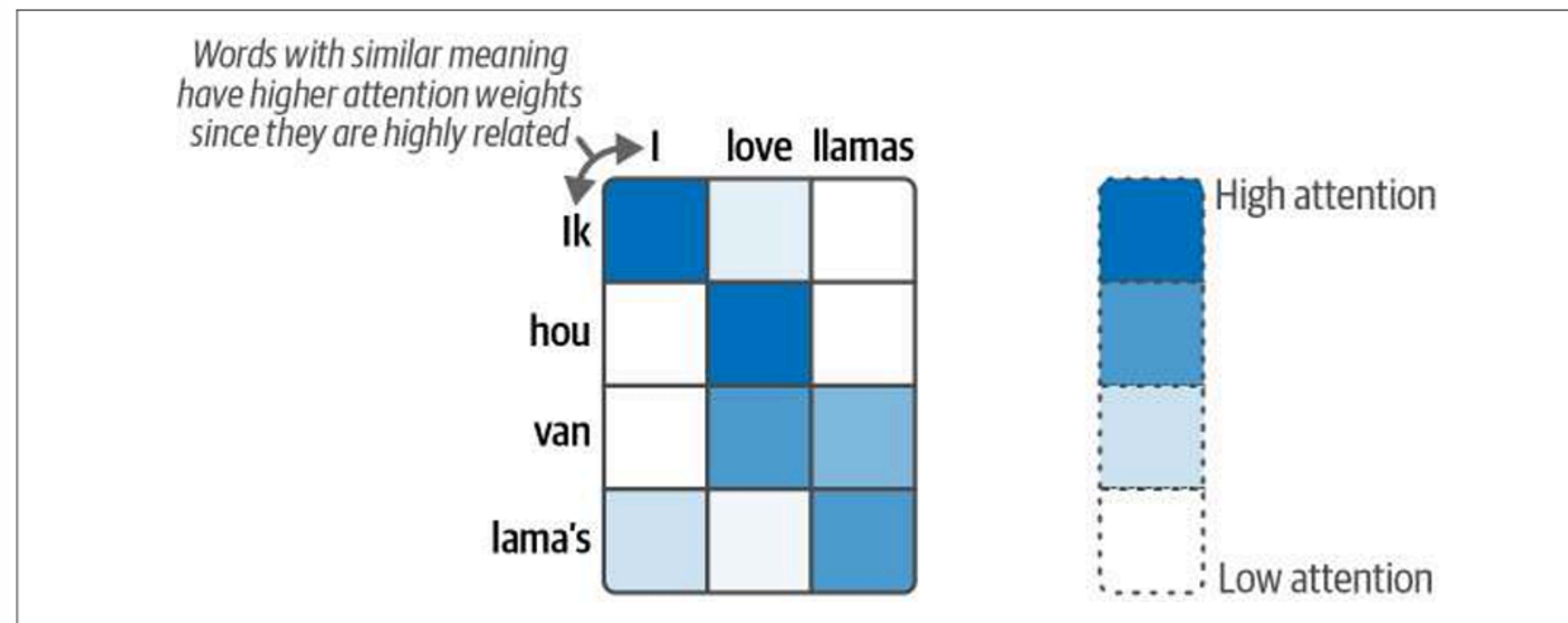


Figure 1-14. Attention allows a model to “attend” to certain parts of sequences that might relate more or less to one another.

By adding these attention mechanisms to the decoder step, the RNN can generate signals for each input word in the sequence related to the potential output. Instead of passing only a context embedding to the decoder, the hidden states of all input words are passed. This process is demonstrated in Figure 1-15.

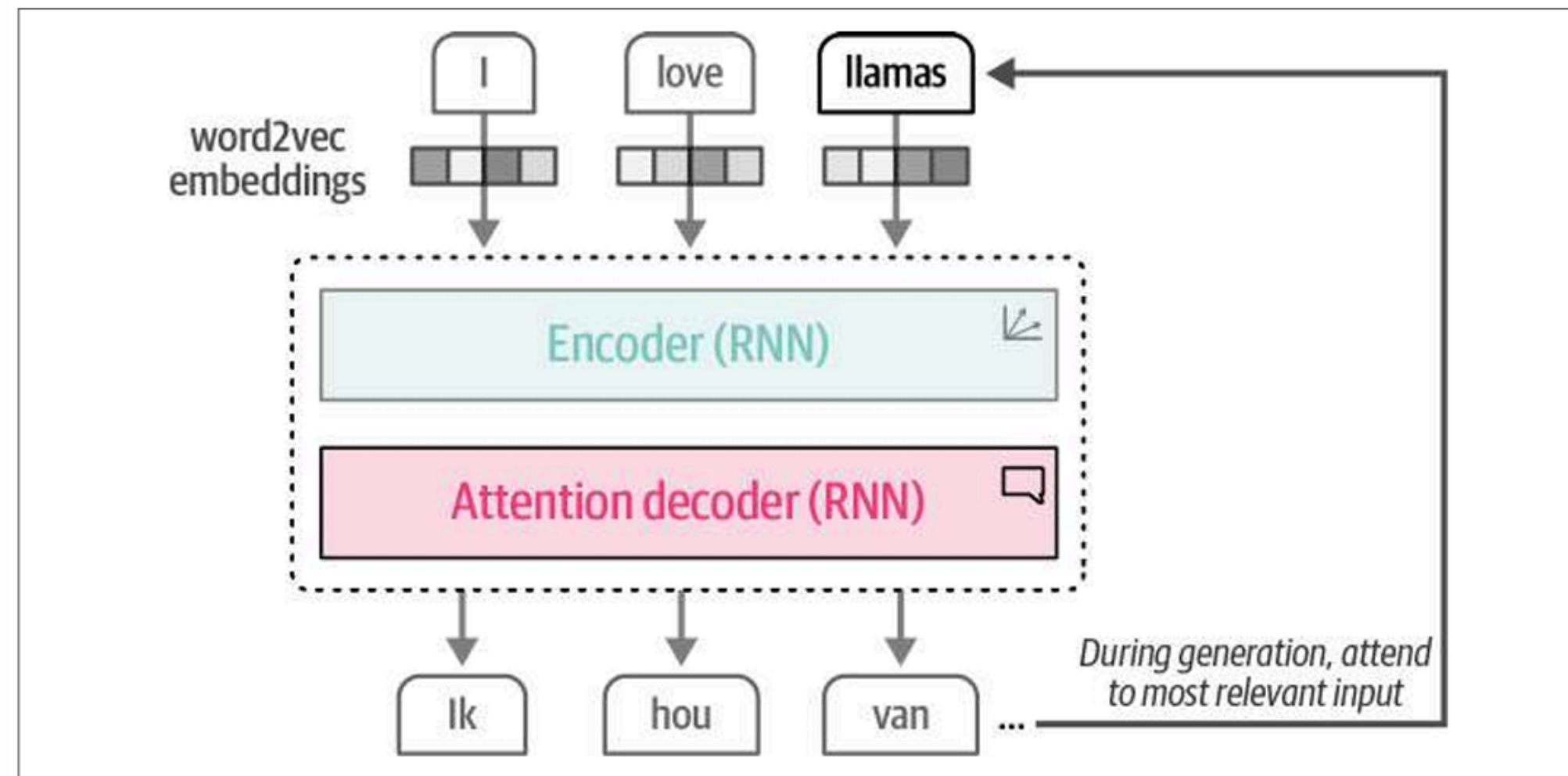
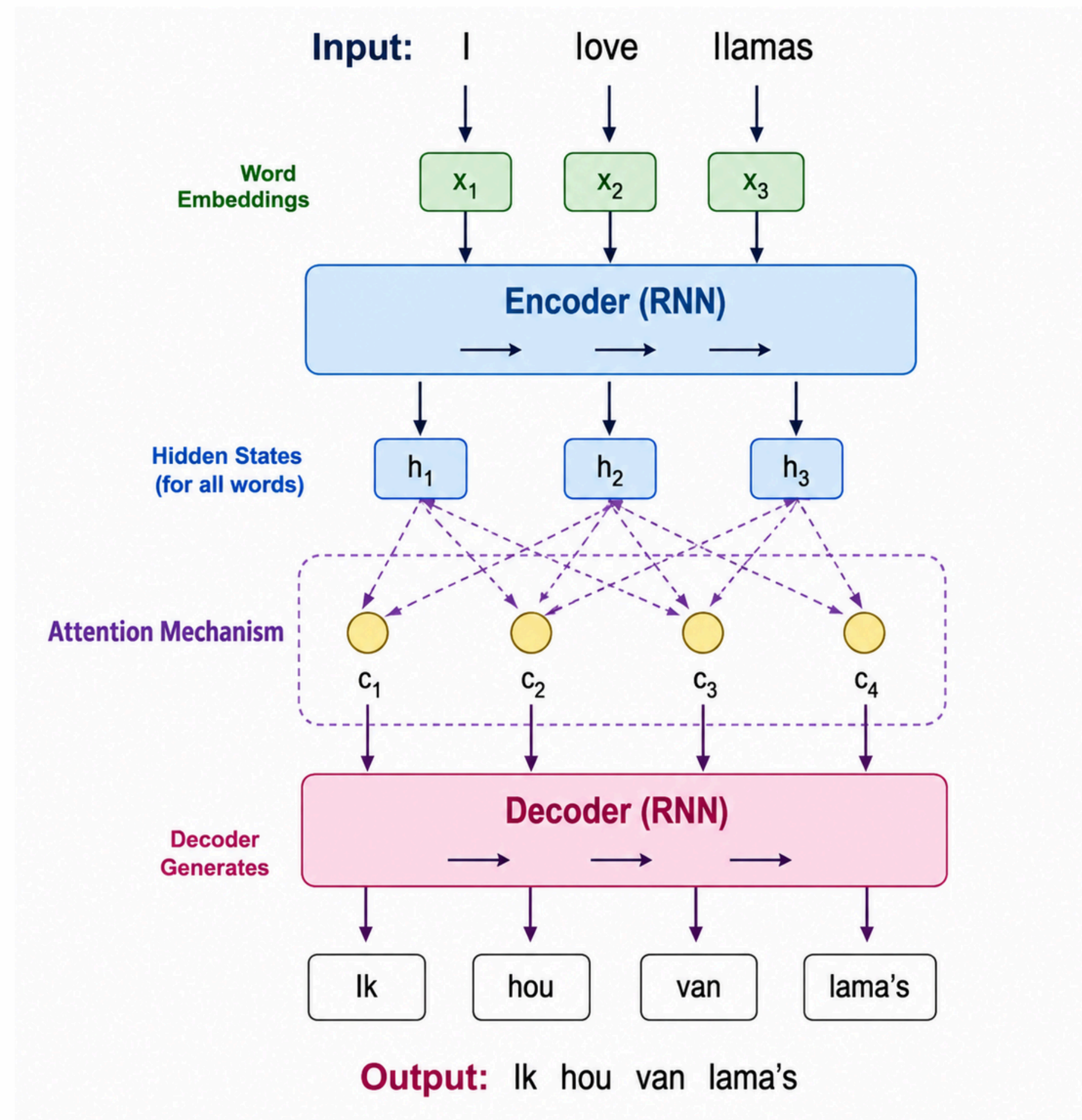


Figure 1-15. After generating the words “Ik,” “hou,” and “van,” the attention mechanism of the decoder enables it to focus on the word “llamas” before it generates the Dutch translation (“lama’s”).



Attention Is All You Need

The true power of attention, and what drives the amazing abilities of large language models, was first explored in the well-known “Attention is all you need” paper released in 2017. The authors proposed a network architecture called the Transformer, which was solely based on the attention mechanism and removed the recurrence network that we saw previously. Compared to the recurrence network, the Transformer could be trained in parallel, which tremendously sped up training. In the Transformer, encoding and decoder components are stacked on top of each other, as illustrated in Figure 1-16. This architecture remains autoregressive, needing to consume each generated word before creating a new word.

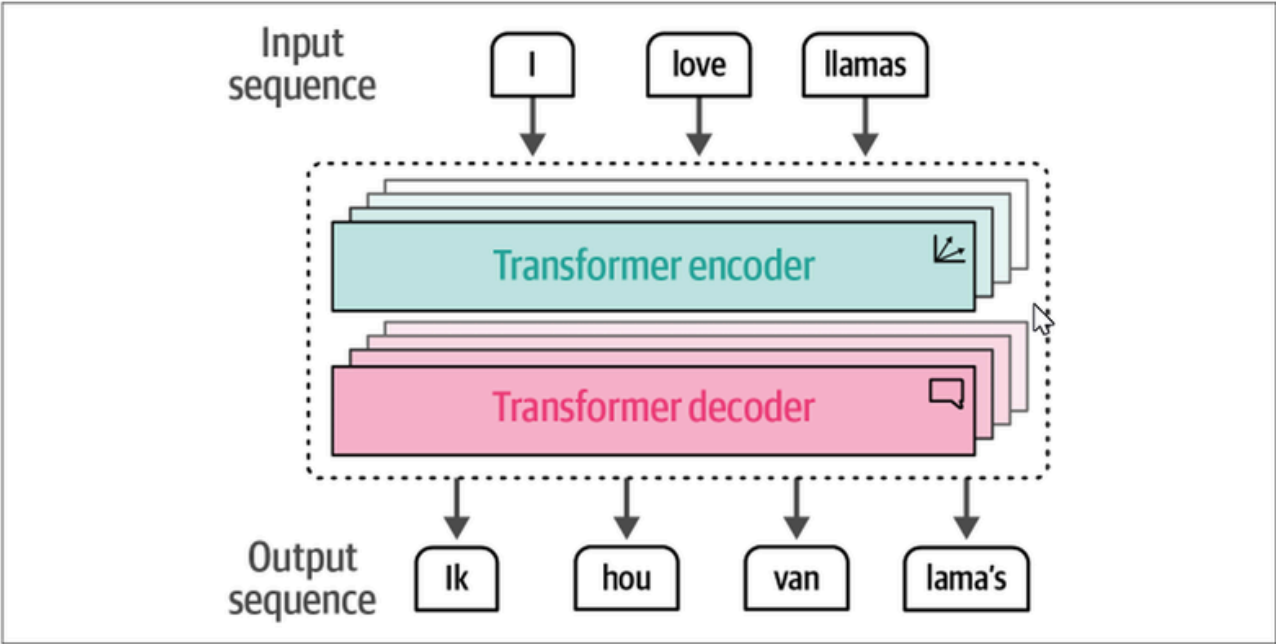
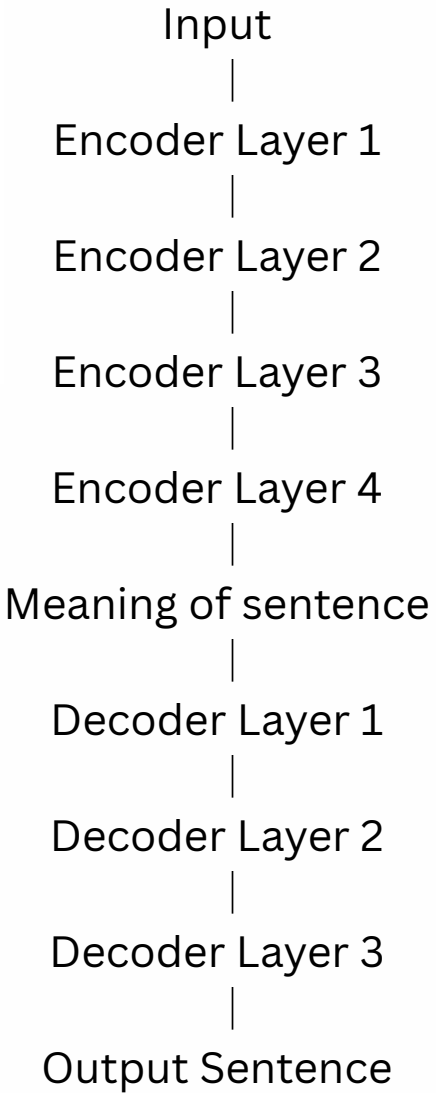


Figure 1-16. The Transformer is a combination of stacked encoder and decoder blocks where the input flows through each encoder and decoder.

Unlike an RNN:
I → love → llamas

the Transformer does
I
love → Together
llamas

Every word can "see" every other word immediately.



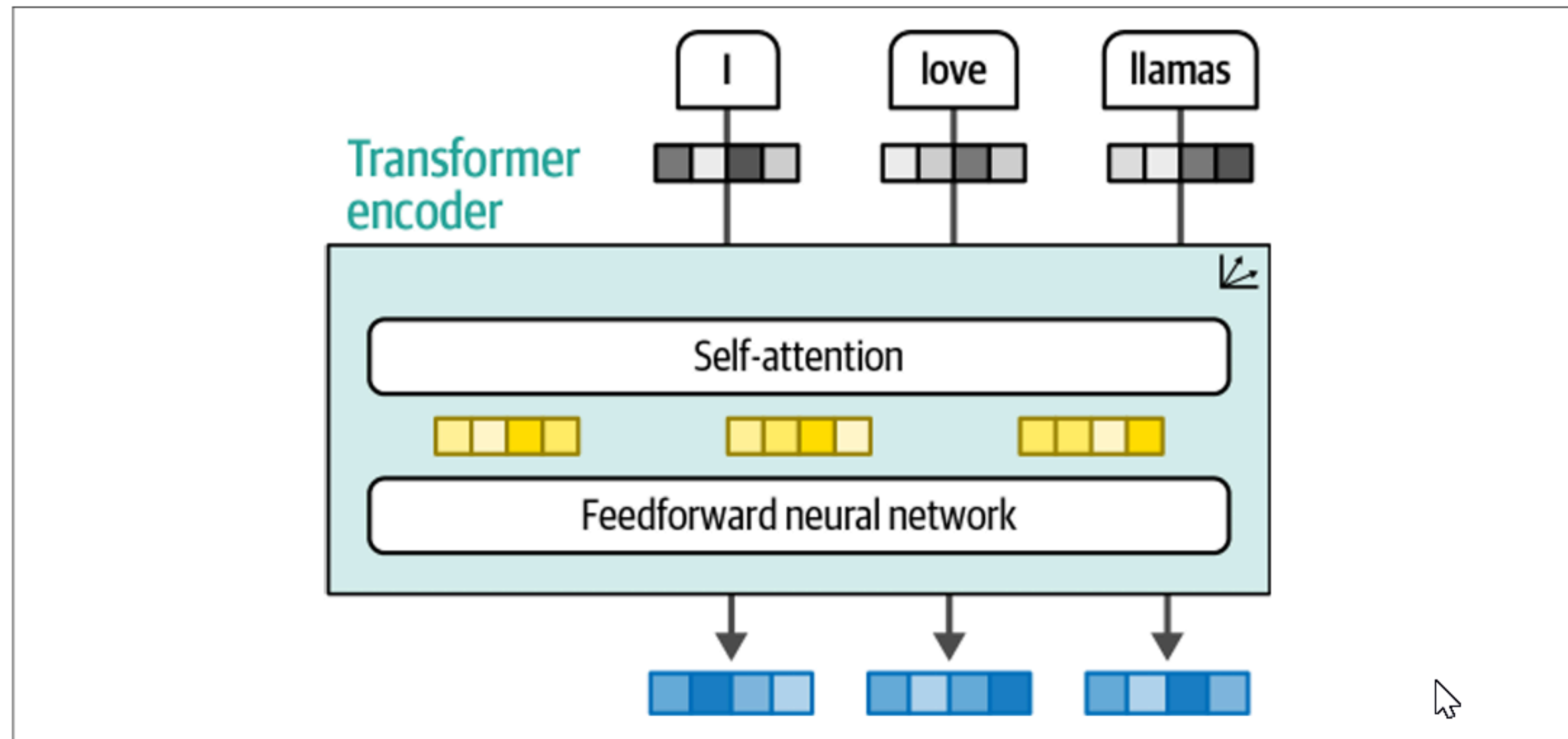


Figure 1-17. An encoder block revolves around self-attention to generate intermediate representations.

Now, both the encoder and decoder blocks would revolve around attention instead of leveraging an RNN with attention features. The encoder block in the Transformer consists of two parts, self-attention and a feedforward neural network, which are shown in Figure 1-17.

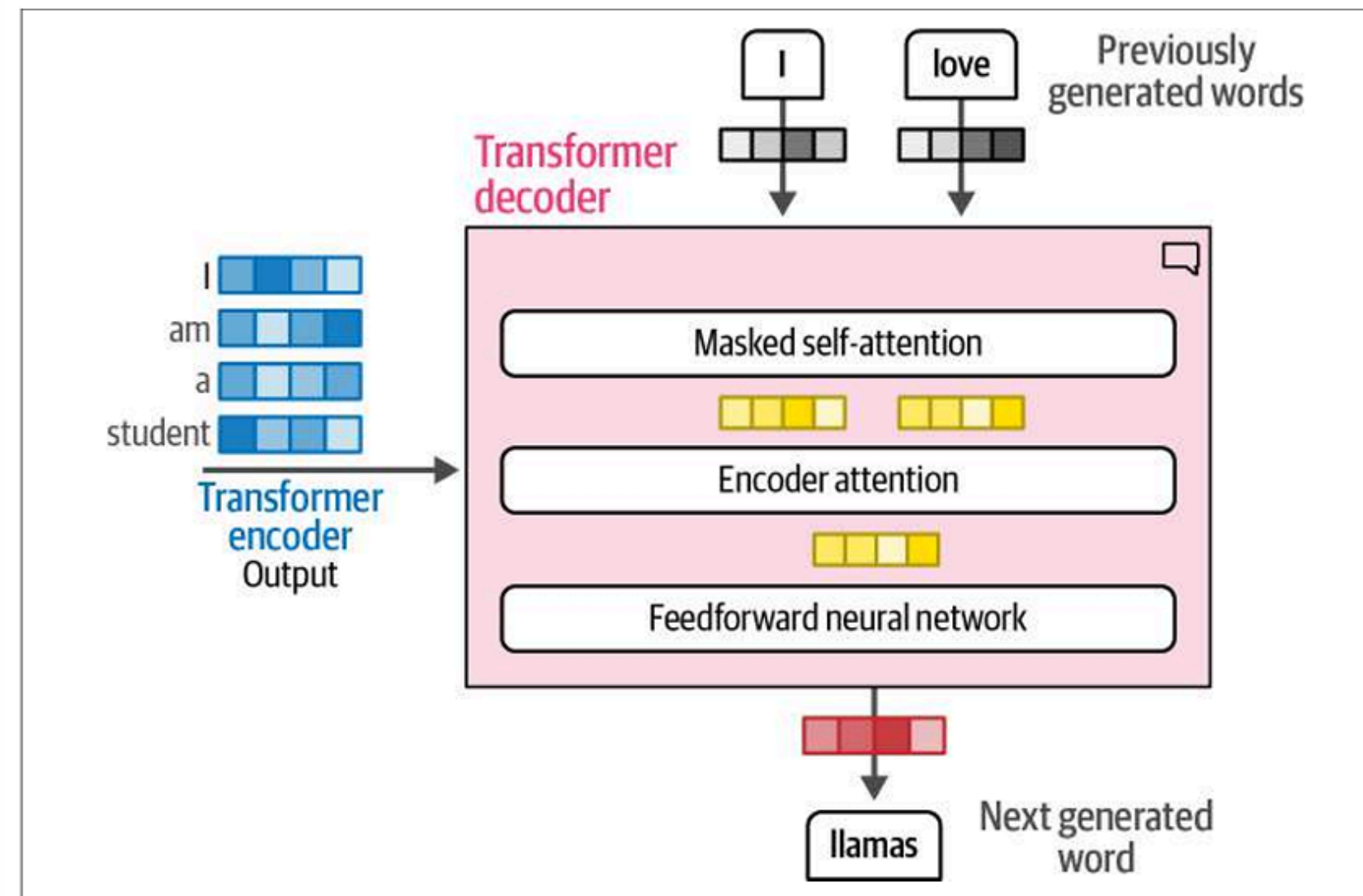


Figure 1-19. The decoder has an additional attention layer that attends to the output of the encoder.

Compared to the encoder, the decoder has an additional layer that pays attention to the output of the encoder (to find the relevant parts of the input). As demonstrated in Figure 1-19, this process is similar to the RNN attention decoder that we discussed previously



Compared to previous methods of attention, self-attention can attend to different positions within a single sequence, thereby more easily and accurately representing the input sequence as illustrated in Figure 1-18. Instead of processing one token at a time, it can be used to look at the entire sequence in one go.

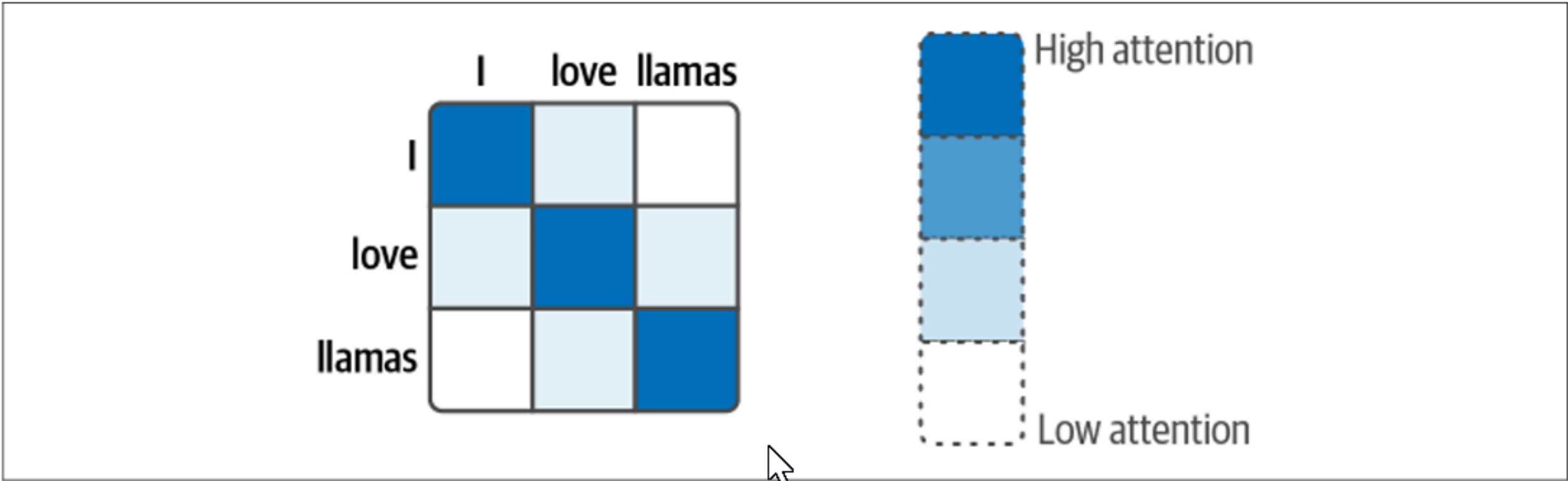


Figure 1-18. Self-attention attends to all parts of the input sequence so that it can “look” both forward and back in a single sequence.

COURSE 1

LARGE LANGUAGE MODELS (LLMs) & AGENTIC AI

